SplitEase

Complete Technical Documentation

Full-stack expense splitting app — architecture, frontend, backend, database, APIs, build process, and deployment guide

VERSION	STACK	DATABASE
1.0.0	Next.js 16 + Prisma 7	SQLite (libSQL)

AUTH
NextAuth v4

TypeScript · Tailwind CSS 4 · Radix UI · Framer Motion · bcryptjs · Zod

Table of Contents

PART 1 — OVERVIEW

1. Project Overview & Goals 3
2. Feature List 3
3. Tech Stack Summary 4

PART 2 — ARCHITECTURE

4. System Architecture 5
   - 4.1 High-level Architecture Diagram 5
   - 4.2 Request Flow 5
   - 4.3 Folder Structure 6

PART 3 — DATABASE

5. Database Design 7
   - 5.1 Schema Overview 7
   - 5.2 Entity Relationship Diagram 7
   - 5.3 All Models Explained 8

PART 4 — BACKEND

6. Authentication System 10
7. API Routes Reference 11
8. Business Logic — Balance Calculation 13

PART 5 — FRONTEND

9. Pages & Components 14
10. UI Component Library 15
11. Routing & Navigation 16

PART 6 — BUILD & DEPLOY

12. Environment Variables 17

13. Build Setup & Commands 17

14. Services & Dependencies 18

15. Deployment Guide (Vercel) 19

1 Project Overview & Goals

SplitEase is a full-stack web application inspired by Splitwise, built to solve the common problem of tracking shared expenses among friends, roommates, and travel groups. The goal was to create a free, modern, self-hostable alternative with a beautiful UI.

Core Problem Solved: When groups of people share expenses (trips, shared housing, dinners), tracking who paid what and who owes whom becomes complex and awkward. SplitEase automates the calculation and simplifies settlement.

Key Goals

💸 Expense Tracking

Log expenses in groups with payer tracking, categories, and date. Support equal splits across all members.

⚖️ Smart Balances

Automatically calculate net balances between all group members, simplifying complex webs of debt into minimum transactions.

👥 Group Management

Create named groups with custom emoji/colour, invite members by email, assign admin roles.

✅ Settle Up

Record settlements that reduce balances. Full history of all settlements per group.

2 Feature List

CATEGORY	FEATURE	STATUS
Auth	Email + password sign up / sign in	✅ Done
Auth	Google OAuth sign in	✅ Done
Auth	JWT session management	✅ Done
Groups	Create group with name, emoji, colour, description	✅ Done

Groups	Invite members by email	✅ Done
Groups	Admin / member roles	✅ Done
Groups	Remove members	✅ Done
Expenses	Add expense with description, amount, category	✅ Done
Expenses	Choose who paid	✅ Done
Expenses	Equal split among all members	✅ Done
Expenses	9 expense categories with icons	✅ Done
Balances	Net balance calculation per group	✅ Done
Balances	Debt simplification algorithm	✅ Done
Settlements	Record settle-up payments	✅ Done
Settlements	Settlement history log	✅ Done
Friends	Send / accept / reject friend requests	✅ Done
Friends	Search users by name / email	✅ Done
Dashboard	Net balance across all groups	✅ Done
Dashboard	Recent activity feed	✅ Done
UI	Dark glassmorphism design	✅ Done
UI	Fully responsive (mobile + desktop)	✅ Done
UI	Animated transitions	✅ Done

3 Tech Stack Summary

LAYER	TECHNOLOGY	VERSION	WHY CHOSEN
Framework	Next.js (App Router)	16.2.3	Full-stack in one repo. Server Components, Server Actions, API routes, file-based routing — eliminates need for separate backend.
Language	TypeScript	5.x	Type safety across frontend + backend prevents runtime bugs. Prisma generates fully-typed models.
Styling	Tailwind CSS	4.x	Utility-first, no context switching. v4 uses PostCSS and CSS variables natively — faster compile times.
UI Components	Radix UI	latest	Headless, accessible primitives (Dialog, Avatar, Tabs, etc.). We style them ourselves for full design control.
Animations	Framer Motion	12.x	Production-grade animations. Used for modal transitions and page elements.
Icons	Lucide React	latest	600+ consistent SVG icons as React components. Tree-shakeable.
ORM	Prisma	7.7.0	Type-safe DB queries. Auto-generates TypeScript types from schema. Supports migrations.
Database	SQLite (via libSQL)	—	Zero-config for local dev. libSQL adapter enables edge/serverless deployment (Turso).
Auth	NextAuth.js	v4	Handles sessions, JWT, OAuth providers, and the Prisma adapter out of the box.
Password Hashing	bcryptjs	3.x	Industry-standard bcrypt hashing — pure JS (no native addons), works everywhere.
Validation	Zod	4.x	Schema-based runtime validation. Integrates with react-hook-form resolvers.
Forms	react-hook-form	7.x	Performant form state management with minimal re-renders.

Class Utilities	clsx + tailwind-merge	latest	Cleanly merge conditional Tailwind classes. `cn()` utility used throughout.
CVA	class-variance-authority	0.7.x	Variant-based component styling (e.g. Button variants: default, outline, ghost).

Package Manager & Tooling

TOOL	PURPOSE
npm 11	Package manager (ships with Node 25)
Node.js 25	Runtime — installed via Homebrew
Turbopack	Next.js 16 built-in bundler for dev (replaces Webpack)
ESLint 9	Linting with next/eslint-config
Homebrew	macOS package manager used to install Node.js
Git	Version control (pre-installed on macOS via Xcode tools)

4 System Architecture

4.1 High-Level Architecture

SplitEase is a **monolithic full-stack app** built on Next.js 16. There is no separate frontend and backend server — everything runs in a single Next.js process. The App Router architecture uses React Server Components for data fetching and Client Components for interactivity.

> **Architecture Pattern:** Server-Side Rendering (SSR) + Client Components + Route Handlers (API). Server Components fetch data directly from the database (no API call overhead). Client Components use fetch() to hit Route Handlers for mutations.

```
┌──────────────────────────────────────────────────────────────────┐
│                        Browser (Client)                          │
│   React Client Components  ──→  fetch("/api/...")  ──→  Server   │
│   Server Components render HTML directly from DB                 │
└──────────────────────────┬───────────────────────────────────────┘
                           │ HTTP
┌──────────────────────────▼───────────────────────────────────────┐
│                    Next.js 16 App Router                         │
│                                                                  │
│  ┌────────────────┐    ┌─────────────────┐    ┌─────────────┐   │
│  │  Page / Layout  │    │   Route Handler   │    │   Proxy     │  │
│  │  (RSC + Client) │    │   /api/...        │    │  (Auth Gate)│  │
│  └────────┬───────┘    └────────┬────────┘    └─────────────┘   │
│           │                     │                                │
│  ┌────────▼─────────────────────▼──────────────────────────┐    │
│  │                    Auth Layer (NextAuth v4)                  │  │
│  │         JWT Session  │  getServerSession()  │  Middleware    │  │
│  └──────────────────────────┬──────────────────────────────┘    │
│                             │                                    │
│  ┌──────────────────────────▼──────────────────────────────┐    │
│  │                   Prisma ORM (v7)                            │  │
│  │         Type-safe query builder  +  Migration engine         │  │
│  └──────────────────────────┬──────────────────────────────┘    │
│                             │                                    │
└─────────────────────────────│────────────────────────────────── ┘
                              │
┌─────────────────────────────▼──────────────┐
│          SQLite Database (via libSQL)          │
│          File: prisma/dev.db                   │
└────────────────────────────────────────────┘
```

4.2 Request Flow

A — Page Load (Server Component)

Browser request → proxy.ts checks JWT → Next.js renders RSC → Prisma queries DB → HTML sent to browser

B — Client Mutation (e.g. Add Expense)

User submits form → fetch POST /api/groups/[id]/expenses → Route Handler validates session → Prisma creates Expense + Splits → JSON response → UI re-fetches

C — Authentication Flow

POST /api/auth/register → bcrypt.hash(password) → prisma.user.create() → signIn("credentials") → JWT issued → cookie set

4.3 Folder Structure

```
splitwise-clone/
├── prisma/
│   ├── schema.prisma          # All DB models
│   ├── dev.db                 # SQLite database file
│   └── migrations/            # Prisma migration files
├── prisma.config.ts           # Prisma 7 config (DB URL, migration path)
├── src/
│   ├── app/                   # Next.js App Router
│   │   ├── layout.tsx         # Root layout (Providers, fonts)
│   │   ├── page.tsx           # Landing page (/)
│   │   ├── globals.css        # Tailwind + custom CSS
│   │   ├── (auth)/            # Route group — no sidebar
│   │   │   ├── layout.tsx     # Centered auth layout
│   │   │   ├── signin/        # /signin page
│   │   │   └── signup/        # /signup page
│   │   ├── (dashboard)/       # Route group — with sidebar
│   │   │   ├── layout.tsx     # Dashboard layout + auth guard
│   │   │   ├── dashboard/     # /dashboard page (overview)
│   │   │   ├── groups/        # /groups list page
│   │   │   │   ├── new/       # /groups/new (create form)
│   │   │   │   └── [id]/      # /groups/[id] (detail page)
│   │   │   └── friends/       # /friends page
```

```
│   │   └── api/               # API Route Handlers
│   │       ├── auth/
│   │       │   ├── [...nextauth]/  # NextAuth handler
│   │       │   └── register/       # POST /api/auth/register
│   │       ├── groups/             # GET, POST
│   │       │   └── [id]/
│   │       │       ├── route.ts    # GET, DELETE group
│   │       │       ├── expenses/   # POST expense
│   │       │       ├── members/    # POST, DELETE member
│   │       │       ├── balances/   # GET balances
│   │       │       └── settle/     # POST settlement
│   │       ├── friends/            # GET, POST friends
│   │       │   └── [id]/          # PATCH (accept/reject)
│   │       └── users/search/       # GET search users
│   ├── components/
│   │   ├── providers.tsx      # SessionProvider wrapper
│   │   ├── sidebar.tsx        # Desktop sidebar + mobile nav
│   │   └── ui/                # Reusable UI components
│   │       ├── button.tsx
│   │       ├── input.tsx
│   │       ├── card.tsx
│   │       ├── avatar.tsx
│   │       ├── dialog.tsx
│   │       └── badge.tsx
│   ├── lib/
│   │   ├── prisma.ts          # Prisma client singleton
│   │   ├── auth.ts            # NextAuth config + callbacks
│   │   └── utils.ts           # cn(), formatCurrency(), etc.
│   ├── types/
│   │   └── next-auth.d.ts     # Session type augmentation (adds user.id)
│   ├── generated/
│   │   └── prisma/            # Auto-generated Prisma client (v7)
│   └── proxy.ts               # Next.js 16 auth proxy / route guard
├── .env                       # Environment variables
├── next.config.ts             # Next.js configuration
├── tsconfig.json              # TypeScript config
└── package.json               # Dependencies
```

5 Database Design

5.1 Overview

The database uses **SQLite** accessed through the **libSQL** driver (compatible with Turso for production). Prisma 7 generates a fully type-safe client from the schema. The schema has 10 models covering users, authentication, groups, expenses, splits, settlements, and friendships.

> **Why SQLite?** Zero configuration for local development — no server to install. The libSQL adapter (Turso) makes it trivially upgradeable to a distributed cloud database when going to production.

5.2 Entity Relationship Diagram

```
User ──────────────────────────────────────────────────────────────
 │ id, name, email, password, image                                 │
 │                                                                   │
 ├──[creates]──→ Group                                               │
 │               │ id, name, description, color, emoji              │
 │               │                                                   │
 │               ├──[has many]──→ GroupMember ←──[belongs to]──────┘
 │               │                  │ userId, role
 │               │                  │
 │               ├──[has many]──→ Expense
 │               │                  │ description, amount, category
 │               │                  │ paidById, splitType, date
 │               │                  │
 │               │                  └──[has many]──→ ExpenseSplit
 │               │                                     │ userId, amount, paid
 │               │
 │               ├──[has many]──→ Settlement
 │               │                  │ fromUserId, toUserId, amount
 │               │
 │               └──[has many]──→ Invitation
 │                                  │ email, token, status
 │
 ├──[OAuth]──→ Account (NextAuth)
 ├──[JWT]───→ Session (NextAuth)
 │
 └──[friends with]──→ Friend
                        │ requesterId, addresseeId, status
```

5.3 All Models Explained

User

FIELD	TYPE	DESCRIPTION
id	String (cuid)	Primary key — collision-safe unique ID
name	String?	Display name (optional for OAuth users)
email	String (unique)	Used for login and member invite
password	String?	bcrypt hash — null for OAuth-only users
image	String?	Profile photo URL (from Google OAuth)
emailVerified	DateTime?	Required by NextAuth Prisma Adapter

Group

FIELD	TYPE	DESCRIPTION
id	String (cuid)	Primary key
name	String	Group display name
description	String?	Optional description
color	String	Hex color for group card (default: #6366f1)
emoji	String	Emoji icon for group (default: 💰)
createdById	FK → User	The user who created the group

GroupMember

FIELD	TYPE	DESCRIPTION
groupId	FK → Group	Part of composite unique key
userId	FK → User	Part of composite unique key
role	String	ADMIN or MEMBER — controls permissions
@@unique([groupId, userId]) prevents duplicate membership		

Expense

FIELD	TYPE	DESCRIPTION
description	String	What the expense was for
amount	Float	Total cost in the group's currency
category	String	food, transport, accommodation, etc.
paidById	FK → User	Who physically paid the bill
splitType	String	EQUAL, EXACT, or PERCENTAGE
date	DateTime	When the expense occurred
currency	String	ISO 4217 code (default: USD)

ExpenseSplit

One record per person per expense. Tracks exactly how much each member owes for a given expense.

FIELD	TYPE	DESCRIPTION
expenseId	FK → Expense	Which expense this split belongs to
userId	FK → User	Which member this split is for
amount	Float	How much this member owes for this expense
paid	Boolean	True if this person was the payer (pre-marked paid)

Settlement

Records a payment made between two members to clear a debt. Immutable — settlements are never deleted, they build up a history.

FIELD	DESCRIPTION
fromUserId	Person making the payment
toUserId	Person receiving the payment
amount	Amount settled
note	Optional memo (e.g. "via Venmo")

Friend

FIELD	DESCRIPTION
requesterId	User who sent the friend request
addresseeId	User who received the request
status	PENDING → ACCEPTED or REJECTED (deleted on reject)
@@unique([requesterId, addresseeId]) prevents duplicate requests	

NextAuth Models (Account, Session, VerificationToken)

These three models are required by the `@auth/prisma-adapter`. They handle OAuth token storage (Account), active sessions (Session), and email verification tokens (VerificationToken). You don't interact with them directly — NextAuth manages them automatically.

6 Authentication System

How It Works

Authentication uses **NextAuth v4** with two providers: Credentials (email + password) and Google OAuth. Sessions are stored as signed **JWT tokens** in a cookie — no server-side session storage required.

Credentials Provider Flow

```
1. User submits email + password to POST /api/auth/register
   → bcrypt.hash(password, 12) — 12 rounds is OWASP recommended
   → prisma.user.create({ email, hashedPassword })
   → Returns user object (no password exposed)

2. User signs in via signIn("credentials", { email, password })
   → NextAuth calls authorize() callback
   → prisma.user.findUnique({ where: { email } })
   → bcrypt.compare(password, user.password)
   → Returns user object → JWT issued with { id, email, name, image }

3. JWT stored in httpOnly cookie (NEXTAUTH_SECRET signs it)
   → Every request: proxy.ts calls getToken({ req }) to validate
   → Server components: getServerSession(authOptions) returns user
```

Session Type Augmentation

NextAuth's default Session type doesn't include `user.id`. This is fixed in `src/types/next-auth.d.ts`:

```
// src/types/next-auth.d.ts
declare module "next-auth" {
  interface Session {
    user: { id: string } & DefaultSession["user"]
  }
}

// src/lib/auth.ts — jwt callback adds id to token
async jwt({ token, user }) {
  if (user) token.id = user.id
  return token
},
// session callback puts it on the session object
```

```
  async session({ session, token }) {
    if (token) session.user.id = token.id
    return session
  }
```

Route Protection — proxy.ts

Next.js 16 renamed `middleware.ts` to `proxy.ts`. The proxy runs on every request before any page renders:

```
// src/proxy.ts
export async function proxy(req: NextRequest) {
  const token = await getToken({ req, secret: process.env.NEXTAUTH_SECRET })

  // Redirect logged-in users away from auth pages
  if (token && ["/", "/signin", "/signup"].includes(pathname))
    return NextResponse.redirect(new URL("/dashboard", req.url))

  // Redirect unauthenticated users to signin
  if (!token && !isPublicPath(pathname))
    return NextResponse.redirect(new URL("/signin", req.url))
}
```

> **Security Note:** Every API Route Handler also independently calls `getServerSession(authOptions)` and returns 401 if no session. The proxy is a UX redirect layer — API security is never proxy-only.

Google OAuth Setup (optional)

To enable Google sign-in, add to `.env`:

```
GOOGLE_CLIENT_ID=your-google-client-id
GOOGLE_CLIENT_SECRET=your-google-client-secret
```

Create credentials at: `console.cloud.google.com → APIs & Services → Credentials → OAuth 2.0 Client`. Set redirect URI to `http://localhost:3000/api/auth/callback/google`.

7 API Routes Reference

All API routes are Next.js Route Handlers located in `src/app/api/`. They use the Web `Request` / `Response` API and return JSON.

> **Next.js 16 Breaking Change:** Route handler `params` is now a **Promise**. You must `await params` before destructuring: `const { id } = await params`.

Authentication Routes

```
POST  /api/auth/register — Create a new user account
```

FIELD	TYPE	NOTES
name	string	Required
email	string	Must be unique
password	string	Hashed with bcrypt (12 rounds)
Returns		201 { id, email, name } | 409 if email taken

```
POST  /api/auth/[...nextauth] — NextAuth handler (sign in, sign out, session)
```

```
GET   /api/auth/[...nextauth] — Get current session
```

Groups Routes

```
GET   /api/groups — List all groups the current user belongs to
```

```
POST  /api/groups — Create a new group (auto-adds creator as ADMIN)
```

BODY FIELD	DESCRIPTION
name (required)	Group display name
description	Optional description

color	Hex color string (default: #6366f1)
emoji	Emoji character (default: 💰)

```
GET    /api/groups/[id] — Get full group with members, expenses, settlements
```

```
DELETE  /api/groups/[id] — Delete group (ADMIN only)
```

Members Routes

```
POST   /api/groups/[id]/members — Add member by email (user must already exist)
```

```
DELETE  /api/groups/[id]/members — Remove a member (ADMIN or self)
```

Expenses Routes

```
POST   /api/groups/[id]/expenses — Add a new expense to the group
```

BODY FIELD	DESCRIPTION
description	What was the expense for
amount	Total amount (number)
category	food | transport | accommodation | entertainment | general | ...
paidById	User ID of the person who paid
splitType	EQUAL | EXACT | PERCENTAGE
splits	Required for EXACT/PERCENTAGE — array of {userId, amount} or {userId, percentage}
date	ISO date string (defaults to now)

Split calculation logic (EQUAL):

```
const perPerson = totalAmount / groupMembers.length
splits = groupMembers.map(m => ({
  userId: m.userId,
  amount: Math.round(perPerson * 100) / 100,  // avoid float precision bugs
```

```
  paid: m.userId === paidById  // payer's split is pre-marked as paid
})))
```

Balances Route

```
GET  /api/groups/[id]/balances — Calculate who owes whom in the group
```

Returns an array of `{ fromUser, toUser, amount }` objects representing simplified net debts. See Section 8 for the full algorithm.

Settlements Route

```
POST  /api/groups/[id]/settle — Record a payment between two members
```

BODY FIELD	DESCRIPTION
toUserId	User receiving the payment
amount	Amount paid
note	Optional memo

Friends Routes

```
GET   /api/friends — List all friend relationships (accepted + pending)
POST  /api/friends — Send a friend request { addresseeId }
PATCH /api/friends/[id] — Accept or reject a request { action: "accept" | "reject" }
```

Users Search Route

```
GET  /api/users/search?q=query — Search users by name or email (min 2 chars)
```

Returns up to 10 users. Excludes the current user. Used for the Add Friend and Add Member flows.

8 Business Logic — Balance Calculation

The balance calculation is the core algorithm of the app. It lives in `GET /api/groups/[id]/balances`.

Step 1 — Build Raw Debt Map

For every **ExpenseSplit** where the split owner ≠ payer and `paid = false`:

```
balances[split.userId][expense.paidById] += split.amount
// e.g. Alice paid $60 dinner, 3-way split:
// Bob owes Alice $20 → balances["bob"]["alice"] += 20
// Carol owes Alice $20 → balances["carol"]["alice"] += 20
```

Step 2 — Apply Settlements

For every Settlement record, subtract from the debtor's balance:

```
balances[settlement.fromUserId][settlement.toUserId] =
  Math.max(0, existing - settlement.amount)
```

Step 3 — Simplify Mutual Debts

If A owes B $30 AND B owes A $20, the net result is A owes B $10 only:

```
for each pair (A, B):
  net = balances[A][B] - balances[B][A]
  if net > 0.01:   emit { fromUser: A, toUser: B, amount: net }
  if net < -0.01:  emit { fromUser: B, toUser: A, amount: -net }
```

> **Result:** The minimum number of transactions needed to settle all debts. A group of 10 people with 50 expenses might need only 5-8 payments to fully clear all balances.

Dashboard Balance (Across All Groups)

The dashboard calculates `totalOwed` (others owe me) and `totalOwe` (I owe others) by scanning all expenses in all groups the user belongs to:

```
if expense.paidById === myId:
  for each split where split.userId !== myId and !split.paid:
    totalOwed += split.amount

if expense.paidById !== myId:
  mySplit = expense.splits.find(s => s.userId === myId)
  if mySplit and !mySplit.paid:
    totalOwe += mySplit.amount

// Subtract confirmed settlements
netBalance = totalOwed - totalOwe
```

9 Pages & Components

Page Inventory

ROUTE	FILE	TYPE	DESCRIPTION
/	app/page.tsx	Static	Landing page — hero, feature highlights, mock dashboard preview, stats, CTA
/signin	app/(auth)/signin/page.tsx	Client	Email/password login form + Google OAuth button
/signup	app/(auth)/signup/page.tsx	Client	Registration form with password strength indicator
/dashboard	app/(dashboard)/dashboard/page.tsx	Server	Net balance cards, recent expenses, groups overview
/groups	app/(dashboard)/groups/page.tsx	Server	Grid of all groups with balance and member count
/groups/new	app/(dashboard)/groups/new/page.tsx	Client	Create group form with emoji/colour picker
/groups/[id]	app/(dashboard)/groups/[id]/page.tsx	Client	Group detail — expenses, balances, members tabs
/friends	app/(dashboard)/friends/page.tsx	Client	Friend search, pending requests, friends list

Layout Hierarchy

```
app/layout.tsx (Root)
  └── Providers (SessionProvider)
        ├── (auth)/layout.tsx  — Centered card, gradient background
        │     ├── /signin
        │     └── /signup
        └── (dashboard)/layout.tsx  — Auth guard + Sidebar + MobileNav
              ├── /dashboard
              ├── /groups
              ├── /groups/new
              ├── /groups/[id]
```

```
            └── /friends
```

> **Route Groups:** `(auth)` and `(dashboard)` are Next.js route groups — the parentheses mean they don't appear in the URL. They exist purely to apply different layouts.

Group Detail Page Architecture

This is the most complex page. It's a **Client Component** that manages its own state and makes API calls. It has three tabs:

TAB	DATA SOURCE	KEY LOGIC
Expenses	GET /api/groups/[id]	Shows net amount per person (you paid → green, you owe → red)
Balances	GET /api/groups/[id]/balances	Simplified debt list. "Settle up" button triggers settlement dialog
Members	GET /api/groups/[id]	List with roles, add member button

Three modal dialogs are embedded:

- **Add Expense dialog** — description, amount, category picker, payer selector
- **Add Member dialog** — email input, server-validates user exists
- **Settle Up dialog** — confirms amount, optional note

10 UI Component Library

All components live in `src/components/ui/`. They are custom-built (not from shadcn/ui) but use Radix UI primitives for accessibility.

Component Reference

Button (`button.tsx`)

Built with `class-variance-authority` for variant support. Supports `loading` prop which shows a spinner and disables the button.

VARIANT	USE CASE
default	Primary action — indigo to purple gradient
destructive	Delete / dangerous actions — rose gradient
outline	Secondary action — glass border
ghost	Tertiary / nav items
success	Settle up / confirm — emerald gradient
link	Inline text links

SIZE	HEIGHT
sm	h-8 (32px)
default	h-10 (40px)
lg	h-12 (48px)
xl	h-14 (56px) — landing page CTAs
icon	h-10 w-10 — square icon buttons

Input (`input.tsx`)

Extends HTML input with `icon` (left-side icon), `error` (red border + message) props. Glassmorphic dark styling with focus ring.

Card (`card.tsx`)

Five sub-components: `Card`, `CardHeader`, `CardTitle`, `CardDescription`, `CardContent`, `CardFooter`. Rounded-2xl, glass border, backdrop blur.

Avatar (`avatar.tsx`)

Wraps Radix Avatar. Fallback shows initials on an indigo-to-purple gradient background. Extracted via `getInitials(name, email)` utility.

Dialog (`dialog.tsx`)

Full-screen modal overlay. Slate-900 background, rounded-2xl, close button, enter/exit animations using Radix data attributes + Tailwind animate classes.

Badge (`badge.tsx`)

Pill-shaped label. Variants: default (indigo), secondary (grey), destructive (rose), success (emerald), warning (amber), outline.

Design System

TOKEN	VALUE	USED FOR
Background	#0a0a1a	Page background
Primary	#6366f1 (indigo-500)	Buttons, focus rings, active nav
Accent	#a855f7 (purple-500)	Gradient end, logo
Success	#22c55e (emerald-500)	Positive balances, settle button
Danger	#f43f5e (rose-500)	Negative balances, delete
Text primary	#f1f5f9 (slate-100)	Headings, important text
Text secondary	#94a3b8 (slate-400)	Labels, descriptions
Border	rgba(255,255,255,0.1)	Card borders, dividers
Glass bg	rgba(255,255,255,0.05)	Cards, inputs, nav
Border radius	rounded-xl (12px) / rounded-2xl (16px)	Cards, buttons, inputs

Utility Functions (`lib/utils.ts`)

FUNCTION	PURPOSE
cn(...classes)	Merge Tailwind classes safely (clsx + tailwind-merge)

formatCurrency(amount, currency)	Intl.NumberFormat — "USD $42.50"
getInitials(name, email)	Returns "JD" from "John Doe" or "J" from email
formatDate(date)	"Apr 14, 2025" via Intl.DateTimeFormat
formatRelativeTime(date)	"2h ago", "3d ago", falls back to formatDate
CATEGORY_ICONS	Map of category → emoji: food→🍔, travel→✈️, etc.
GROUP_EMOJIS	Array of 10 selectable group emoji
GROUP_COLORS	Array of 10 selectable hex colors

11 Routing & Navigation

Sidebar (Desktop)

The `Sidebar` component is a **Client Component** (uses `usePathname()` for active state and `useSession()` for the user card). It renders on all `(dashboard)` routes. Features:

- SplitEase logo and branding
- New Group quick-action button
- Three nav items: Dashboard, Groups, Friends (with active state highlight)
- User avatar + name + email at bottom
- Sign out button (calls `signOut({ callbackUrl: "/" })`)

Mobile Navigation

The `MobileNav` component renders as a fixed bottom tab bar on screens under `md` breakpoint (768px). Shows icon + label for Dashboard, Groups, Friends, and New group.

Navigation Guard

Two layers of protection:

1. **proxy.ts** — Edge-level check using `getToken()`. Redirects unauthenticated requests to `/signin` before the page even renders.
2. **Dashboard layout** — Server-side `getServerSession()` check with `redirect("/signin")` as a fallback in case proxy is bypassed.

12 Environment Variables

All environment variables go in `.env` at the project root. **Never commit this file.**

VARIABLE	REQUIRED	DESCRIPTION
DATABASE_URL	Yes	`file:./prisma/dev.db` for local SQLite. Change to Turso URL for production.
NEXTAUTH_SECRET	Yes	Random string used to sign JWTs. Generate with: `openssl rand -base64 32`
NEXTAUTH_URL	Yes	`http://localhost:3000` locally. Your production domain in prod.
GOOGLE_CLIENT_ID	Optional	From Google Cloud Console — enables Google OAuth login
GOOGLE_CLIENT_SECRET	Optional	From Google Cloud Console

13 Build Setup & Commands

Prerequisites

REQUIREMENT	VERSION	HOW TO INSTALL
Node.js	≥ 18 (used 25)	`brew install node`
npm	≥ 9 (used 11)	Ships with Node.js
Git	any	Pre-installed on macOS

Commands

COMMAND	WHAT IT DOES
`npm install`	Install all dependencies from package.json
`npx prisma db push`	Sync schema to DB without migrations (dev only). Creates `dev.db` if missing.

`npx prisma generate`	Regenerate the Prisma client after schema changes. Output: `src/generated/prisma/`
`npm run dev`	Start dev server with Turbopack on `http://localhost:3000`
`npm run build`	Production build (type-checks, compiles, optimises)
`npm run start`	Serve the production build
`npm run lint`	Run ESLint checks

Fresh Setup (from scratch)

```
# 1. Clone / navigate to project
cd splitwise-clone

# 2. Install dependencies
npm install

# 3. Configure environment
cp .env.example .env
# Edit .env — set NEXTAUTH_SECRET and DATABASE_URL

# 4. Create database and tables
npx prisma db push

# 5. Generate Prisma client
npx prisma generate

# 6. Start development server
npm run dev
# Open http://localhost:3000
```

14 Services & Dependencies Explained

PACKAGE	VERSION	ROLE	WHY IT MATTERS
next	16.2.3	Core framework	App Router, RSC, Route Handlers, Turbopack bundler
react / react-dom	19.2.4	UI runtime	Server + Client Components, hooks, transitions

prisma	7.7.0	ORM CLI	Schema management, db push, migrations, client generation
@prisma/client	7.7.0	DB query client	Type-safe database access. In v7, requires a driver adapter.
@prisma/adapter-libsql	7.7.0	DB adapter	Connects Prisma to libSQL/SQLite. Needed in Prisma 7 — the old direct connection is deprecated.
@libsql/client	latest	SQLite driver	Pure-JS SQLite client that works in Node.js and edge runtimes. Compatible with Turso cloud.
next-auth	4.24.x	Authentication	Handles OAuth, credentials, JWT sessions, and CSRF protection out of the box.
@auth/prisma-adapter	2.x	Auth ↔ DB bridge	Tells NextAuth to store sessions/accounts in our Prisma database.
bcryptjs	3.x	Password hashing	Pure JS bcrypt — no native build required. 12 rounds ≈ OWASP recommendation.
tailwindcss	4.x	CSS framework	v4 uses a new CSS-first config system. No tailwind.config.js needed — config lives in globals.css.
@tailwindcss/postcss	4.x	PostCSS plugin	Required by Tailwind v4 to process CSS during the build.
@radix-ui/*	latest	Accessible primitives	Headless components (Dialog, Avatar, Tabs, etc.) with full keyboard/ARIA support.
framer-motion	12.x	Animations	Dialog open/close animations, hover effects, layout transitions.
lucide-react	latest	Icons	600+ SVG icons as React components. Individual imports keep bundle small.
class-variance-authority	0.7.x	Component variants	Type-safe variant definition for Button (default, outline, destructive, etc.).
clsx	2.x	Class merging	Conditionally apply class strings without string concat mess.

tailwind-merge	3.x	Class conflict resolution	Merges Tailwind classes intelligently (e.g. last `bg-*` wins).
zod	4.x	Schema validation	Runtime type checking for API inputs and form data.
react-hook-form	7.x	Form state	Performant forms with minimal re-renders. Used with Zod resolver.
dotenv	17.x	Env loading	Required by `prisma.config.ts` to load `.env` before Prisma CLI runs.
nodemailer	7.x	Email (future)	Installed for future invite email functionality.

15 Deployment Guide (Vercel)

> **Recommended:** Deploy to Vercel (built by the Next.js team) + Turso (managed libSQL database). Both have generous free tiers.

Step 1 — Set up Turso database

```
# Install Turso CLI
brew install tursodatabase/tap/turso

# Sign in / sign up
turso auth login

# Create a database
turso db create splitease-prod

# Get the connection URL and auth token
turso db show splitease-prod --url
turso db tokens create splitease-prod
```

Step 2 — Update schema for Turso

Turso uses libSQL which is SQLite-compatible. Update `.env` (production):

```
DATABASE_URL=libsql://splitease-prod-[your-org].turso.io
TURSO_AUTH_TOKEN=your-token-here
```

Update `src/lib/prisma.ts` to use the auth token for production:

```
const adapter = new PrismaLibSql({
  url: process.env.DATABASE_URL!,
  authToken: process.env.TURSO_AUTH_TOKEN,
})
```

Step 3 — Deploy to Vercel

```
# Install Vercel CLI
npm install -g vercel

# Deploy (follow prompts)
vercel

# Set production environment variables
vercel env add DATABASE_URL
vercel env add TURSO_AUTH_TOKEN
vercel env add NEXTAUTH_SECRET
vercel env add NEXTAUTH_URL         # Set to https://your-app.vercel.app
vercel env add GOOGLE_CLIENT_ID     # If using Google OAuth
vercel env add GOOGLE_CLIENT_SECRET

# Deploy to production
vercel --prod
```

Step 4 — Run migrations on production DB

```
# Point CLI at production DB temporarily
DATABASE_URL=libsql://... npx prisma db push
```

Alternative: Self-hosted with Docker

```
# Build Docker image
docker build -t splitease .

# Run with SQLite (volume-mounted)
docker run -p 3000:3000 \
  -e NEXTAUTH_SECRET=your-secret \
```

```
  -e DATABASE_URL=file:/data/prod.db \
  -v splitease-data:/data \
  splitease
```

Production Checklist

ITEM	STATUS
Generate strong NEXTAUTH_SECRET (`openssl rand -base64 32`)	☐
Set NEXTAUTH_URL to production domain	☐
Switch DATABASE_URL to Turso/PostgreSQL	☐
Update Google OAuth redirect URI to production URL	☐
Run `prisma db push` on production database	☐
Enable HTTPS (automatic on Vercel)	☐
Test sign up, group creation, expense, settle flow end-to-end	☐

SplitEase Technical Documentation — v1.0.0
Built with Next.js 16, Prisma 7, NextAuth v4, Tailwind CSS 4, SQLite (libSQL)